Cours 4 : Agréger des données avec GROUP BY et HAVING

CHAFAOUI Mohamed-Reda

CF2M

15 avril 2026

Plan du cours

1. Pourquoi agréger des données ?
2. Les fonctions d'agrégation
3. GROUP BY
4. Différence entre WHERE et HAVING
5. Ordre des clauses
6. Erreurs fréquentes
7. Exercices progressifs

Pourquoi agréger des données ?

Idée générale

Agréger des données consiste à résumer plusieurs lignes pour obtenir une information synthétique.

- Combien de commandes avons-nous ?
- Quel est le chiffre d'affaires total ?
- Quelle est la note moyenne d'un produit ?
- Quel est le prix minimum ou maximum ?
- Combien de commandes par client ou par statut ?

Important

Sans agrégation : on lit les lignes une par une.
Avec agrégation : on obtient une vue globale ou par groupe.

Exemple visuel

Table `orders`

id	customer_id	status	total_amount
1	1	paid	120
2	1	paid	80
3	2	pending	200
4	2	paid	150
5	3	cancelled	90

- `COUNT(*)` : nombre total de lignes
- `SUM(total_amount)` : somme
- `AVG(total_amount)` : moyenne
- `GROUP BY customer_id` : un résultat par client

Les fonctions d'agrégation

Fonction	Rôle	Exemple
`COUNT()`	Compter	`COUNT(*)`
`SUM()`	Additionner	`SUM(total_amount)`
`AVG()`	Moyenne	`AVG(rating)`
`MIN()`	Minimum	`MIN(price)`
`MAX()`	Maximum	`MAX(price)`

À retenir

Sans `GROUP BY`, on obtient généralement un seul résultat global.

COUNT

Rappel

`COUNT(*)` compte toutes les lignes.
`COUNT(colonne)` ignore les valeurs `NULL`.

```sql
SELECT COUNT(*) AS nb_commandes
FROM orders;
```

```sql
SELECT COUNT(email) AS nb_emails
FROM customers;
```

SUM

```sql
SELECT SUM(total_amount) AS chiffre_affaires
FROM orders
WHERE status = 'paid';
```

```sql
SELECT SUM(quantity) AS quantite_totale
FROM order_items;
```

AVG

```sql
SELECT AVG(rating) AS note_moyenne
FROM reviews;
```

```sql
SELECT AVG(total_amount) AS panier_moyen
FROM orders
WHERE status = 'paid';
```

MIN et MAX

```sql
SELECT MIN(price) AS prix_minimum
FROM products;
```

```sql
SELECT MAX(price) AS prix_maximum
FROM products;
```

```sql
SELECT
    MIN(created_at) AS premiere_commande,
    MAX(created_at) AS derniere_commande
FROM orders;
```

GROUP BY

Principe

`GROUP BY` regroupe les lignes qui ont la même valeur sur une ou plusieurs colonnes.

- Sans `GROUP BY` : résultat global
- Avec `GROUP BY` : résultat par groupe

Premier exemple

```sql
SELECT
    status,
    COUNT(*) AS nb_commandes
FROM orders
GROUP BY status;
```

CA par client

```sql
SELECT
    customer_id,
    SUM(total_amount) AS chiffre_affaires
FROM orders
WHERE status = 'paid'
GROUP BY customer_id;
```

Note moyenne par produit

```sql
SELECT
    product_id,
    AVG(rating) AS note_moyenne
FROM reviews
GROUP BY product_id;
```

```sql
SELECT
    product_id,
    AVG(rating) AS note_moyenne
FROM reviews
GROUP BY product_id
ORDER BY note_moyenne DESC;
```

Règle importante

Règle

Avec `GROUP BY`, chaque colonne du `SELECT` doit être :

- soit dans le `GROUP BY`
- soit dans une fonction d'agrégation

Exemple correct / incorrect

Correct

```sql
SELECT customer_id, SUM(total_amount)
FROM orders
GROUP BY customer_id;
```

Incorrect

```sql
SELECT id, customer_id, SUM(total_amount)
FROM orders
GROUP BY customer_id;
```

GROUP BY sur plusieurs colonnes

```sql
SELECT
    customer_id,
    status,
    COUNT(*) AS nb_commandes
FROM orders
GROUP BY customer_id, status;
```

WHERE vs HAVING

Clause	Agit sur	Moment
`WHERE`	les lignes	avant le regroupement
`HAVING`	les groupes	après le regroupement

À retenir

`WHERE` filtre les lignes.
`HAVING` filtre les groupes.

Exemple avec WHERE

```sql
SELECT
    customer_id,
    COUNT(*) AS nb_commandes
FROM orders
WHERE status = 'paid'
GROUP BY customer_id;
```

Exemple avec HAVING

```sql
SELECT
    customer_id,
    COUNT(*) AS nb_commandes
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 2;
```

WHERE et HAVING ensemble

```sql
SELECT
    customer_id,
    SUM(total_amount) AS chiffre_affaires
FROM orders
WHERE status = 'paid'
GROUP BY customer_id
HAVING SUM(total_amount) > 500;
```

Erreur classique

Incorrect

```
SELECT customer_id, COUNT(*)
FROM orders
WHERE COUNT(*) > 2
GROUP BY customer_id;
```

Correct

```
SELECT customer_id, COUNT(*)
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 2;
```

Ordre des clauses

1. SELECT
2. FROM
3. WHERE
4. GROUP BY
5. HAVING
6. ORDER BY
7. LIMIT

Exemple complet

```sql
SELECT
    customer_id,
    SUM(total_amount) AS chiffre_affaires
FROM orders
WHERE status = 'paid'
GROUP BY customer_id
HAVING SUM(total_amount) > 100
ORDER BY chiffre_affaires DESC
LIMIT 5;
```

Erreurs fréquentes

1. Oublier `GROUP BY`
2. Mettre une colonne non agrégée dans le `SELECT`
3. Confondre `WHERE` et `HAVING`
4. Oublier que `COUNT(colonne)` ignore les `NULL`
5. Faire un calcul sur de mauvaises lignes

Exercices progressifs

Méthode

Toujours se poser 3 questions :

1. Quelles lignes je garde ?
2. Résultat global ou par groupe ?
3. Filtre avant ou après agrégation ?

Exercice 1

Énoncé

Afficher le nombre total de commandes.

```sql
SELECT COUNT(*) AS nb_commandes
FROM orders;
```

Exercice 2

Énoncé

Afficher le chiffre d'affaires total des commandes payées.

```sql
SELECT SUM(total_amount) AS chiffre_affaires
FROM orders
WHERE status = 'paid';
```

Exercice 3

Énoncé

Afficher le nombre de commandes par statut.

```sql
SELECT
    status,
    COUNT(*) AS nb_commandes
FROM orders
GROUP BY status;
```

Exercice 4

Énoncé

Afficher le chiffre d'affaires par client pour les commandes payées.

```sql
SELECT
    customer_id,
    SUM(total_amount) AS chiffre_affaires
FROM orders
WHERE status = 'paid'
GROUP BY customer_id;
```

Exercice 5

Énoncé

Afficher la note moyenne par produit.

```sql
SELECT
    product_id,
    AVG(rating) AS note_moyenne
FROM reviews
GROUP BY product_id;
```

Exercice 6

Énoncé

Afficher les clients qui ont passé plus de 3 commandes.

```sql
SELECT
    customer_id,
    COUNT(*) AS nb_commandes
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 3;
```

Exercice 7

Énoncé

Afficher les produits dont la note moyenne est supérieure ou égale à 4.

```sql
SELECT
    product_id,
    AVG(rating) AS note_moyenne
FROM reviews
GROUP BY product_id
HAVING AVG(rating) >= 4;
```

Exercice 8

Énoncé

Afficher les clients dont le chiffre d'affaires dépasse 500 euros sur les commandes payées.

```sql
SELECT
    customer_id,
    SUM(total_amount) AS chiffre_affaires
FROM orders
WHERE status = 'paid'
GROUP BY customer_id
HAVING SUM(total_amount) > 500;
```

Récapitulatif

- ✓ `COUNT` compte
- ✓ `SUM` additionne
- ✓ `AVG` calcule une moyenne
- ✓ `MIN` et `MAX` trouvent les extrêmes
- ✓ `GROUP BY` regroupe
- ✓ `WHERE` filtre les lignes
- ✓ `HAVING` filtre les groupes

Idée clé

Filtrer, regrouper, agréger, puis filtrer les groupes si nécessaire.

Questions ?

Place à la pratique !